

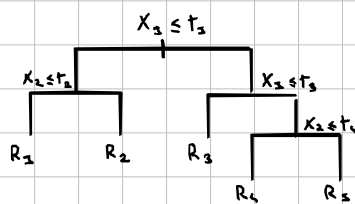
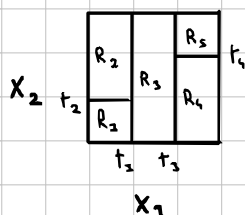
Tree-based Models

CART (Classification and Regression Trees)

A way to predict a continuous output Y from inputs $X_1, X_2, \dots$

Using recursive binary splitting, CART always makes one axis-aligned cut.

Process produces a set of rectangular regions $R_1, \dots, R_m$ where m is the number of leaves.



⚠ Tree generalises to more features we just can't draw it in 2D space.

Prediction mathematically: $\hat{f}(X) = \sum_{m=1}^M c_m \mathbb{I}\{(X_1, X_2) \in R_m\}$

Finding the splits

We want each group we form to have values close to each other.

At any cut we want to split by choosing:

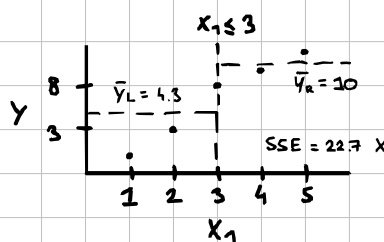
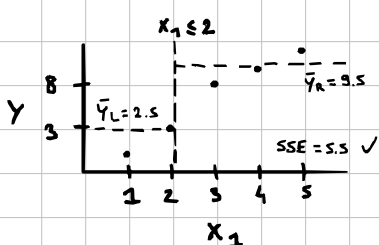
1. Which feature to split on ($X_1, X_2, \dots, X_m$)
2. Which threshold t to split on (e.g. $X_1 \leq t$)

For this we find which pair minimises:

$$SSE_{split} = \sum_{i \in R_L} (y_i - \bar{y}_L)^2 + \sum_{i \in R_R} (y_i - \bar{y}_R)^2$$

where: R_L and R_R regions created by split.

$\bar{y}$, mean of prediction (so avg. Y in region).



⚠ Here we would choose split $X_1 \leq 2$

How big should the tree be?

Naive idea - split if SSE gain exceeds a threshold.

Problem - a split that looks useless might unlock a very good one after.

Preferred strategy: grow big, then prune.

1. Grow large tree T_0 , stopping only when nodes are small (< 5 examples)
2. Prune back using cost-complexity pruning.

Cost-Complexity Pruning

For each leaf m in subtree T define:

N_m = number of examples in m

$\bar{c}_m$ = mean of y_i in leaf m

$$Q_m(T) = \frac{1}{N_m} \sum_{x_i \in R_m} (y_i - \bar{c}_m)^2 \text{ (within leaf variance)}$$

The Cost-Complexity criterion:

$$C_\alpha(T) = \underbrace{\sum_{m=1}^{|T|} N_m Q_m(T)}_{\text{training error}} + \underbrace{\alpha |T|}_{\text{complexity penalty}}$$

→ Then at each pruning collapse the node for which we have smallest increase in training error.

→ This produces a finite subset of trees with decreasing complexity:

$$T_0 \supset T_1 \supset \dots \supset \text{root}$$